

REMIO

Plataforma de Serviços Domésticos

ARQUITETURA, SEGURANÇA E QUALIDADE DA PLATAFORMA

Observabilidade · Segurança · Resiliência · IA

Trabalho de Conclusão de Curso

MBA em Gestão Analítica com BI e Big Data

Universidade de Fortaleza — UNIFOR

Fortaleza — CE, 2026

SUMÁRIO

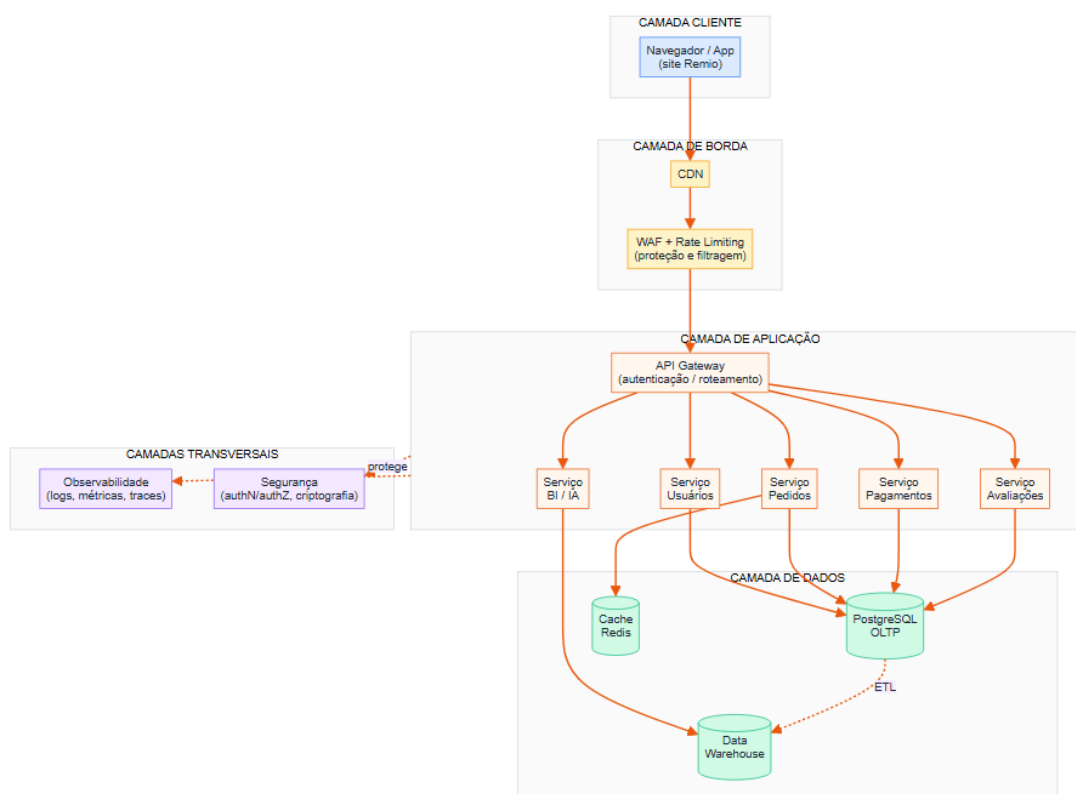
1 Introdução e Arquitetura-Alvo.....	3
2 Estilo Arquitetural: Monólito Modular e Microserviços.....	5
3 Segurança em Camadas.....	6
3.1 Proteção contra SQL Injection.....	7
4 Observabilidade.....	8
5 Resiliência do Software.....	9
6 Gestão de Backups e Recuperação de Desastres.....	10
7 Qualidade e Verificação Contínua.....	11
8 Spec-Driven Development e CI/CD.....	12
9 Repositório de Erros e Correção.....	13
10 Fluxos de Inteligência Artificial.....	14
11 Considerações Finais.....	15

1 INTRODUÇÃO E ARQUITETURA-ALVO

Este documento descreve a arquitetura-alvo da plataforma Remio sob a ótica de engenharia de software: como o sistema seria estruturado em produção para ser seguro, observável, resiliente e de alta qualidade. Enquanto o protótipo entregue é uma aplicação front-end com dados simulados, esta seção projeta o sistema completo que o sustentaria em escala real, atendendo aos requisitos não funcionais da plataforma.

A arquitetura adota uma organização em camadas, com responsabilidades bem separadas e duas camadas transversais — observabilidade e segurança — que permeiam todo o sistema.

Figura 1 — Arquitetura-alvo da plataforma em camadas.



Fonte: elaborado pelos autores (2026).

As camadas se dividem assim:

- **Cliente:** o site/app que o usuário acessa.
- **Borda:** CDN para desempenho e WAF (Web Application Firewall) com rate limiting para barrar tráfego malicioso antes que chegue à aplicação.

- **Aplicação:** um API Gateway que autentica e roteia as requisições para os serviços de negócio (usuários, pedidos, pagamentos, avaliações e BI/IA).
- **Dados:** o banco transacional (OLTP), o Data Warehouse e um cache (Redis) para acelerar leituras frequentes.
- **Transversais:** observabilidade e segurança, aplicadas a todas as camadas anteriores.

2 ESTILO ARQUITETURAL: MONÓLITO MODULAR E MICROSERVIÇOS

Uma decisão central de arquitetura é escolher entre um monólito (aplicação única) e microsserviços (vários serviços independentes). A Remio adota uma estratégia evolutiva, adequada ao estágio do negócio.

Abordagem	Vantagens	Quando usar na Remio
Monólito modular	Simples de desenvolver, testar e implantar; menor custo operacional.	Fase inicial — poucos usuários, equipe pequena, validação do modelo de negócio.
Microsserviços	Escala independente por serviço; isolamento de falhas; times autônomos.	Fase de crescimento — quando o volume e a equipe justificarem a complexidade.

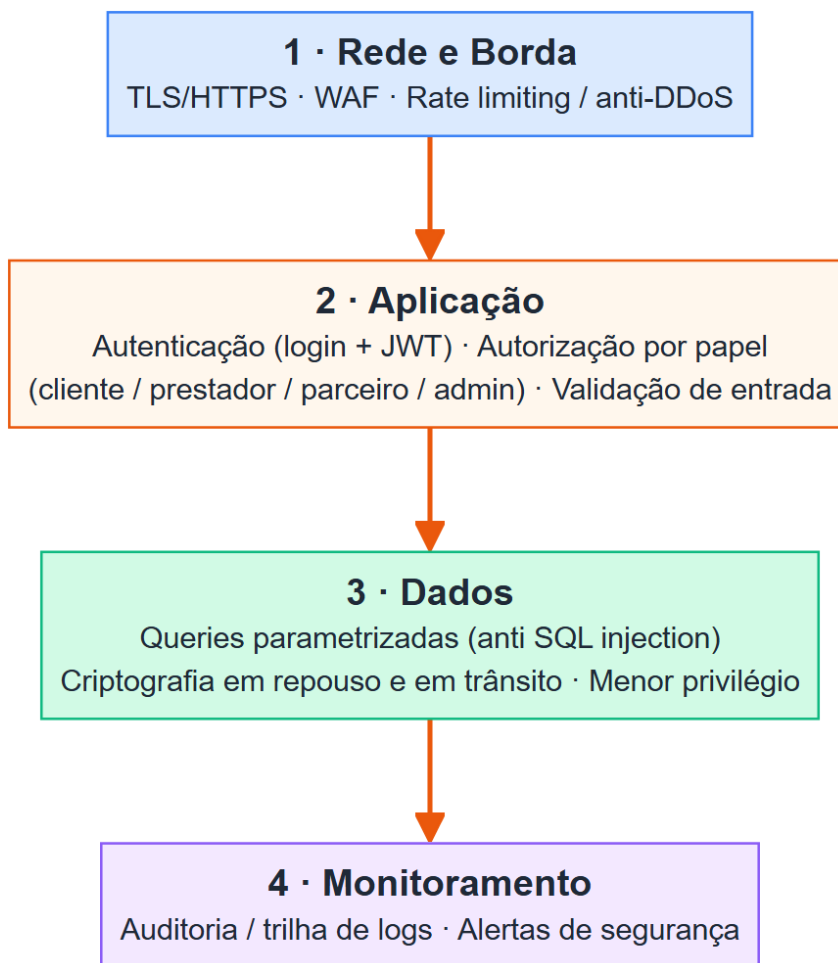
Decisão adotada: iniciar como um monólito modular — uma aplicação única, porém internamente organizada em módulos bem separados (usuários, pedidos, pagamentos, avaliações), com fronteiras claras. Essa organização permite, no futuro, extrair cada módulo como um microsserviço sem reescrever o sistema. Adotar microsserviços desde o início, numa plataforma em validação, seria over-engineering: traria complexidade de rede, orquestração e monitoramento desproporcional ao porte atual.

Essa decisão dialoga diretamente com o plano de negócios, que projeta uma empresa em formação e crescimento gradual: a arquitetura acompanha a maturidade da operação.

3 SEGURANÇA EM CAMADAS

A segurança da Remio segue o princípio de defesa em profundidade (defense in depth): em vez de uma única barreira, várias camadas de proteção se sobrepõem, de modo que a falha de uma não comprometa todo o sistema.

Figura 2 — Defesa em profundidade: quatro camadas de segurança.



Fonte: elaborado pelos autores (2026).

Cada camada cumpre um papel:

- **Rede e borda:** todo o tráfego é cifrado por TLS/HTTPS; o WAF filtra ataques conhecidos; o rate limiting impede abuso e ataques de negação de serviço.
- **Aplicação:** autenticação por login com tokens JWT; autorização por papel (cada perfil só acessa o que lhe compete); e validação rigorosa de toda entrada do usuário.

- **Dados:** queries parametrizadas contra SQL injection, criptografia dos dados sensíveis e o princípio do menor privilégio (cada serviço usa um usuário de banco com permissões mínimas).
- **Monitoramento:** trilha de auditoria de todas as ações sensíveis e alertas automáticos diante de comportamentos suspeitos.

3.1 Proteção contra SQL Injection

O SQL injection é um dos ataques mais comuns: o atacante insere comandos SQL em campos de entrada para manipular a consulta. A defesa definitiva são as queries parametrizadas (prepared statements), nas quais os dados do usuário nunca são interpretados como código. O contraste abaixo ilustra o problema e a solução.

Código vulnerável (concatenação de texto — nunca faça isto):

```
// A entrada do usuário vira parte do comando SQL:  
const sql = "SELECT * FROM usuario WHERE email = '" + entrada + "'";  
  
// Entrada maliciosa: ' OR '1'='1  
// SQL resultante: ... WHERE email = '' OR '1'='1'  
// Resultado: retorna TODOS os usuários (vazamento de dados).
```

Código seguro (query parametrizada — padrão adotado na Remio):

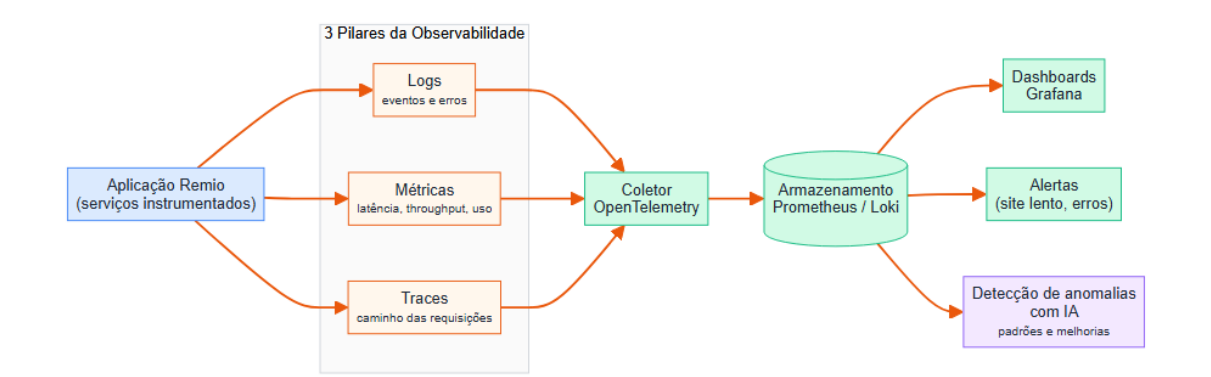
```
// O valor é enviado separado do comando, como parâmetro:  
const sql = "SELECT * FROM usuario WHERE email = $1";  
await db.query(sql, [entrada]);  
  
// A entrada nunca é interpretada como SQL — apenas como dado.  
// O ataque deixa de funcionar.
```

Os exemplos de consulta do documento de Engenharia de Dados já seguem esse padrão, usando parâmetros nomeados (por exemplo, :id) em vez de concatenar valores.

4 OBSERVABILIDADE

Observabilidade é a capacidade de entender o que acontece dentro do sistema a partir do que ele emite. Vai além do monitoramento tradicional: permite identificar automaticamente padrões, gargalos e oportunidades de melhoria — exatamente o tipo de análise que o orientador destacou. Apoia-se em três pilares.

Figura 3 — Os três pilares da observabilidade e o fluxo até alertas e IA.



Fonte: elaborado pelos autores (2026).

Pilar	O que registra	Pergunta que responde
Logs	Eventos e erros, com contexto.	O que aconteceu e por quê?
Métricas	Números ao longo do tempo (latência, throughput, uso de CPU).	O sistema está saudável e rápido?
Traces	O caminho completo de uma requisição entre serviços.	Onde está o gargalo?

Os três sinais são coletados por um agente padronizado (OpenTelemetry), armazenados (Prometheus para métricas, Loki para logs) e visualizados em dashboards (Grafana). Sobre essa base atuam os alertas automáticos (por exemplo, "latência acima de 2s" ou "taxa de erro subindo") e uma camada de detecção de anomalias com IA, capaz de apontar padrões e sugerir melhorias sem intervenção manual.

5 RESILIÊNCIA DO SOFTWARE

Resiliência é a capacidade do sistema de continuar funcionando — ou degradar de forma controlada — diante de falhas. Em vez de supor que tudo sempre funciona, projeta-se para a falha. Os principais padrões adotados:

Padrão	Função
Retry com backoff	Repete automaticamente uma operação que falhou de forma transitória, com intervalos crescentes.
Circuit breaker	Interrompe chamadas a um serviço que está falhando, evitando efeito dominó, e testa a recuperação depois.
Timeout	Define um tempo máximo de espera, impedindo que uma requisição travada bloqueie recursos.
Redundância	Múltiplas instâncias de cada serviço e réplicas do banco, sem ponto único de falha.
Degradação graciosa	Diante de falha parcial, o sistema oferece uma versão reduzida (ex.: dashboard mostra o último dado válido em vez de quebrar).
Filas de mensagens	Desacoplam serviços; se um está fora do ar, as mensagens aguardam e são processadas depois.

Esses padrões são medidos por metas de nível de serviço (SLO — Service Level Objectives), como "99,5% de disponibilidade mensal", que orientam tanto o desenvolvimento quanto os alertas de observabilidade.

6 GESTÃO DE BACKUPS E RECUPERAÇÃO DE DESASTRES

Os dados são o ativo mais crítico da plataforma. A estratégia de backup combina cópias regulares com a capacidade de restaurar o banco a qualquer ponto no tempo, minimizando perdas.

Mecanismo	Descrição
Backup completo (pg_dump)	Cópia integral do banco PostgreSQL, executada diariamente em horário de baixa demanda.
PITR (Point-in-Time Recovery)	Arquivamento contínuo dos logs de transação (WAL), permitindo restaurar o banco para qualquer instante — por exemplo, segundos antes de um erro.
Retenção em camadas	Backups diários por 30 dias, semanais por 3 meses e mensais por 1 ano.
Armazenamento externo	Cópias replicadas em local geograficamente distinto, protegendo contra desastres físicos.
Testes de restauração	Restauração periódica em ambiente isolado para garantir que os backups realmente funcionam.

Duas métricas guiam a estratégia: o **RPO** (Recovery Point Objective — quanto de dado se aceita perder, alvo de minutos com PITR) e o **RTO** (Recovery Time Objective — em quanto tempo o sistema volta, alvo de poucas horas).

7 QUALIDADE E VERIFICAÇÃO CONTÍNUA

Para responder à preocupação levantada pelo orientador — "como saber se o site está ficando lento ou se os painéis estão com dados errados?" — a plataforma adota rotinas automáticas de verificação em três frentes.

Saúde do sistema (o site está lento?)

- **Health checks:** cada serviço expõe um endpoint de saúde verificado continuamente; se não responder, dispara alerta e aciona a redundância.
- **Monitoramento sintético:** robôs simulam a jornada do usuário (buscar profissional, abrir orçamento) em intervalos regulares e medem o tempo de resposta, detectando lentidão antes do usuário real.

Qualidade dos dados (os painéis estão certos?)

Painéis errados quase sempre nascem de dados ruins na carga. Por isso, o ETL ganha etapas de validação de qualidade de dados (data quality) antes de publicar no Data Warehouse:

- Verificação de completude: nenhuma chave estrangeira órfã, nenhum campo obrigatório nulo.
- Verificação de consistência: somatórios conferem (ex.: `valor_comissao + valor_prestador = valor_total`).
- Verificação de volume: a carga do dia está dentro da faixa esperada (uma queda brusca indica falha na extração).
- Em caso de falha, a carga é interrompida e um alerta é emitido — o painel nunca exibe dado inválido.

Testes automatizados

Testes unitários, de integração e de ponta a ponta rodam a cada alteração de código (ver seção 8), garantindo que novas funcionalidades não quebrem as existentes.

8 SPEC-DRIVEN DEVELOPMENT E CI/CD

O desenvolvimento da Remio seguiu a abordagem Spec-Driven Development (desenvolvimento guiado por especificação): antes de escrever código, define-se uma especificação clara do que será construído. Este próprio projeto foi conduzido assim — cada parte partiu de um documento de design (DESIGN.md) validado antes da implementação.

Figura 4 — Ciclo do spec-driven development integrado ao CI/CD.



Fonte: elaborado pelos autores (2026).

O ciclo completo funciona como um fluxo contínuo:

- **Especificação → Plano → Código:** a ideia é detalhada, decomposta em tarefas e então implementada.
- **Testes → Build → Deploy (CI/CD):** a cada alteração, uma esteira automatizada roda os testes, monta a aplicação (Integração Contínua) e a publica (Entrega Contínua), reduzindo erro humano.
- **Monitoramento → Melhoria:** a observabilidade em produção gera aprendizados que retroalimentam novas especificações, fechando o ciclo.

Benefício para o TCC: os documentos de design e os registros de prompts já produzidos são a evidência concreta de que o projeto adotou essa metodologia, e não um desenvolvimento improvisado.

9 REPOSITÓRIO DE ERROS E CORREÇÃO

Um repositório de erros centraliza, registra e prioriza as falhas que ocorrem em produção, transformando-as em correções. Em vez de depender de o usuário relatar um problema, o próprio sistema captura e cataloga cada erro.

Etapa	O que acontece
Captura	Toda exceção (no front-end e no back-end) é interceptada e enviada a uma ferramenta de rastreamento (ex.: Sentry).
Agrupamento	Erros idênticos são agrupados, com contagem de ocorrências, usuários afetados e contexto (navegador, rota, versão).
Priorização	Os erros são ordenados por impacto (frequência × severidade), guiando o time sobre o que corrigir primeiro.
Correção e verificação	A correção entra pelo fluxo de CI/CD; o erro é marcado como resolvido e o sistema confirma se parou de ocorrer.

No protótipo, esse conceito será demonstrado de forma simplificada: uma tela que captura erros de JavaScript do navegador e os exibe em uma lista — uma versão didática do repositório de erros.

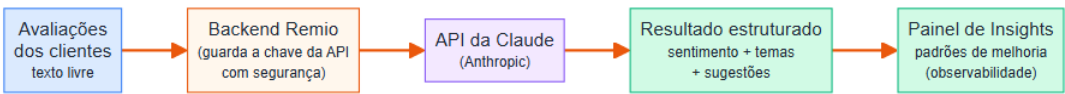
10 FLUXOS DE INTELIGÊNCIA ARTIFICIAL

A Inteligência Artificial amplia o valor da plataforma em várias frentes. Os principais casos de uso previstos para a Remio:

Caso de uso	Benefício
Análise das avaliações	Classifica o sentimento e extrai temas recorrentes dos comentários, revelando padrões de melhoria.
Detecção de fraude	Identifica transações suspeitas no Pagamento Protegido em tempo real.
Previsão de demanda	Antecipa picos por categoria e região, orientando a captação de prestadores.
Matching inteligente	Recomenda ao cliente os prestadores com maior chance de atender bem ao pedido.
Detecção de anomalias	Apoia a observabilidade, sinalizando comportamentos fora do padrão.

O primeiro caso — análise das avaliações — foi efetivamente implementado neste trabalho, usando a API da Claude (Anthropic). O fluxo recebe os comentários dos clientes e devolve uma análise estruturada (sentimento, temas e sugestões de melhoria), conectando-se à observabilidade como uma fonte automática de insights.

Figura 5 — Fluxo de IA: das avaliações à análise estruturada pela API da Claude.



Fonte: elaborado pelos autores (2026).

Boa prática de segurança no fluxo de IA: a chave da API nunca fica no código do site (que é público). A chamada à Claude passa por um backend, que guarda a chave com segurança — o mesmo princípio de menor privilégio e proteção de segredos descrito na seção de segurança. Os detalhes da implementação constam na documentação técnica do repositório.

11 CONSIDERAÇÕES FINAIS

Este documento elevou a Remio de um protótipo funcional para uma plataforma pensada de ponta a ponta sob critérios profissionais de engenharia: arquitetura em camadas com evolução planejada de monólito a microsserviços, segurança em profundidade, observabilidade nos três pilares, resiliência a falhas, gestão de backups, verificação contínua de saúde e de qualidade de dados, metodologia spec-driven com CI/CD, rastreamento de erros e uso aplicado de Inteligência Artificial.

Em conjunto, esses elementos demonstram maturidade arquitetural e respondem diretamente às melhorias sugeridas pelo orientador, fortalecendo a fundamentação técnica do trabalho. O protótipo permanece como prova de conceito da experiência do usuário e da camada de BI, enquanto esta arquitetura define o caminho para a operação real e escalável da plataforma.